

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное
программное обеспечение
Дисциплина «Вычислительная математика»

Лабораторная работа №1

Метод Гаусса

Выполнил

Храбров Артём Алексеевич

Проверил

Наумова Надежда Александровна

Санкт-Петербург, 2026

Цель

Изучить численные методы решения систем линейных алгебраических уравнений и реализовать один из них средствами программирования.

Описание метода

Метод Гаусса. Пример 1

Рассмотрим алгоритм решения линейной системы методом Гаусса для случая трех уравнений:

$$\begin{cases} 10x_1 - 7x_2 = 7 \\ -3x_1 + 2x_2 + 6x_3 = 4 \\ 5x_1 - x_2 + 5x_3 = 6 \end{cases}$$

Исключим x_1 из второго и третьего уравнений. Для этого сначала умножим первое уравнение на 0.3 и результат прибавим ко второму, а затем умножим первое же уравнение на -0.5 и результат прибавим к третьему. Получим:

$$\begin{cases} 10x_1 - 7x_2 = 7 \\ -0.1x_2 + 6x_3 = 6.1 \\ 2.5x_2 + 5x_3 = 2.5 \end{cases}$$

Исключим x_2 из третьего уравнения (заметьте, что ведущий элемент a_{22} мал, поэтому было бы лучше переставить второе и третье уравнения). Однако мы проводим сейчас вычисления в рамках точной арифметики и погрешности округлений не опасны, поэтому продолжим исключение. Умножим второе уравнение на 25 и результат сложим с третьим уравнением. Получим систему в треугольном виде:

$$\begin{cases} 10x_1 - 7x_2 = 7 \\ -0.1x_2 + 6x_3 = 6.1 \\ 155x_3 = 155 \end{cases}$$

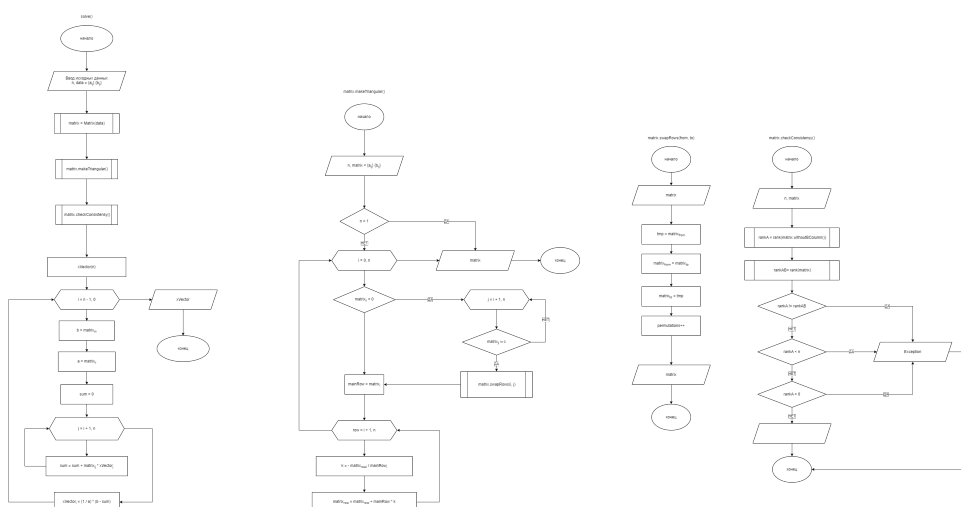
На этом заканчивается прямой ход метода Гаусса.

Обратный ход состоит в последовательном вычислении x_3, x_2, x_1

$$x_3 = \frac{155}{155} = 1 \quad x_2 = \frac{6x_3 - 6.1}{0.1} = -1 \quad x_1 = \frac{7x_2 + 7}{10} = 0$$

Подстановкой в исходную систему легко убедиться, что $(0, -1, 1)$ и есть ее решение.

Блок-схема алгоритма



Листинг программы

Класс с методами для решения системы уравнений

```
1 package com.example.math.models
2
3 class Matrix(val matrix: MutableList<MatrixRow>) {
4     val size = matrix.size
5     private var permutations = 0
6
7     constructor(matrix: List<MutableList<Double>>, tmp: String = "") :
8     this(
9         matrix.map { MatrixRow(it) }.toMutableList()
10    )
11
12    fun print() {
13        matrix.forEach { it.print() }
14    }
15
16    private fun swapRows(from: Int, to: Int) {
17        val tmp = matrix[from]
18        matrix[from] = matrix[to]
19        matrix[to] = tmp
20        permutations++
21    }
22
23    fun makeTriangular() {
24        if (size == 1) return
25
26        for (i in 0 until size) {
27
28            if (matrix[i][i] == 0.0) {
29                for (j in i + 1 until size) {
30                    if (matrix[j][i] != 0.0) {
31                        swapRows(i, j)
32                        break
33                    }
34                }
35            }
36
37            val mainRow = matrix[i]
38
39            for (row in i + 1 until size) {
40                val k = -matrix[row][i] / mainRow[i]
41                matrix[row] = matrix[row] + (mainRow * k)
42            }
43        }
44    }
45 }
```

```

44
45 fun solve(): List<Double> {
46     val matrixCopy = copy()
47     matrixCopy.makeTriangular()
48     matrixCopy.checkConsistency()
49
50     val xVector = MutableList(size) { 0.0 }
51
52     for (i in size - 1 downTo 0) {
53         val b = matrixCopy.matrix[i][size]
54         val a = matrixCopy.matrix[i][i]
55
56         var sum = 0.0
57         for (j in i + 1 until size) {
58             sum += matrixCopy.matrix[i][j] * xVector[j]
59         }
60
61         xVector[i] = (b - sum) / a
62     }
63
64     return xVector
65 }
66
67 fun determinant(): Double {
68     val matrixCopy = copy()
69     matrixCopy.makeTriangular()
70
71     val sign = if (matrixCopy.permutations % 2 == 0) 1.0 else -1.0
72     var det = sign
73
74     for (i in 0 until size) {
75         det *= matrixCopy.matrix[i][i]
76     }
77     return det
78 }
79
80 fun discrepancy(): List<Double> {
81     val xVector = solve()
82     val disVector = MutableList(size) { 0.0 }
83
84     for (i in 0 until size) {
85         var sum = 0.0
86         for (j in 0 until size) {
87             sum += xVector[j] * matrix[i][j]
88         }
89         disVector[i] = sum - matrix[i][size]
90     }
91     return disVector

```

```

92     }
93
94     fun checkConsistency() {
95         val rankA = rank(withoutBColumn())
96         val rankAB = rank(this)
97
98         when {
99             rankAB != rankA -> throw IllegalStateException("Matrix has 0
100             solutions")
101             rankA < size -> throw IllegalStateException("Matrix has
102             infinite solutions")
103             rankA == 0 -> throw IllegalStateException("Matrix has all
104             zeros")
105         }
106     }
107 }
108
109 private fun rank(matrix: Matrix): Int {
110     val matrixCopy = matrix.copy()
111     matrixCopy.makeTriangular()
112
113     return matrixCopy.matrix.count { !it.isNullable() }
114 }
115
116 private fun withoutBColumn(): Matrix {
117     val newRows = matrix.map { row ->
118         MatrixRow(row.row.dropLast(1).toMutableList())
119     }.toMutableList()
120     return Matrix(newRows)
121 }
122
123 fun copy(): Matrix {
124     val newRows = matrix.map { row ->
125         MatrixRow(row.row.toMutableList())
126     }.toMutableList()
127     return Matrix(newRows)
128 }
129
130 companion object {
131     fun randomMatrix(n: Int): Matrix {
132         val rows: MutableList<MatrixRow> = mutableListOf()
133         (0 until n).forEach { _ -> rows.add(MatrixRow.randomRow(n +
134         1)) }
135
136         return Matrix(rows)
137     }
138 }
139 }

```

Пример работы программы

```
MATRIX SOLVER

▶ Source matrix
| 1      2      5      7 |
| 7      8     12      3 |
| 3      3      7      1 |

▶ Triangular matrix
| 1      2      5      7 |
| 0     -6    -23    -46 |
| 0      0     3,5      3 |

Determinant: -21

▶ x_vector:
x1 = -6,047619047619047
x2 = 4,380952380952381
x3 = 0,857142857142857

▶ discrepancy vector:
r1 = 0,0000000000000001
r2 = 0
r3 = 0

Apache math solution:

▶ x_vector:
x1 = -6,047619047619049
x2 = 4,380952380952382
x3 = 0,857142857142857

Determinant: -21

Exit
Retry
```

Ссылка на репозиторий с реализацией

Ссылка

Вывод

В ходе лабораторной работы я изучил принцип работы метода Гаусса, являющегося прямым методом решения СЛАУ. Написал программу, позволяющую читать матрицу из файла или консоли, вычисляющую

с помощью метода Гаусса решение СЛАУ, детерминант, а также вектор невязок.